

Practical 1

Bitonic Sort — Design and Analysis of Algorithms

Program (C)

```
// Aim: implement Bitonic Sort
#include <stdio.h>

void compareAndSwap(int arr[], int i, int j, int dir) {
    if ((dir == 1 && arr[i] > arr[j]) || (dir == 0 && arr[i] < arr[j])) {
        int t = arr[i]; arr[i] = arr[j]; arr[j] = t;
    }
}

void bitonicMerge(int arr[], int low, int cnt, int dir) {
    if (cnt > 1) {
        int k = cnt / 2;
        for (int i = low; i < low + k; i++)
            compareAndSwap(arr, i, i + k, dir);
        bitonicMerge(arr, low, k, dir);
        bitonicMerge(arr, low + k, k, dir);
    }
}

void bitonicSort(int arr[], int low, int cnt, int dir) {
    if (cnt > 1) {
        int k = cnt / 2;
        bitonicSort(arr, low, k, 1);
        bitonicSort(arr, low + k, k, 0);
        bitonicMerge(arr, low, cnt, dir);
    }
}

void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {3, 7, 4, 8, 6, 2, 1, 5};
    int n = sizeof(arr) / sizeof(arr[0]);

    printf("Original array: ");
    printArray(arr, n);

    bitonicSort(arr, 0, n, 1);

    printf("Sorted array:  ");
    printArray(arr, n);

    return 0;
}
```

Output

```
Original array: 3 7 4 8 6 2 1 5
Sorted array:  1 2 3 4 5 6 7 8
```